



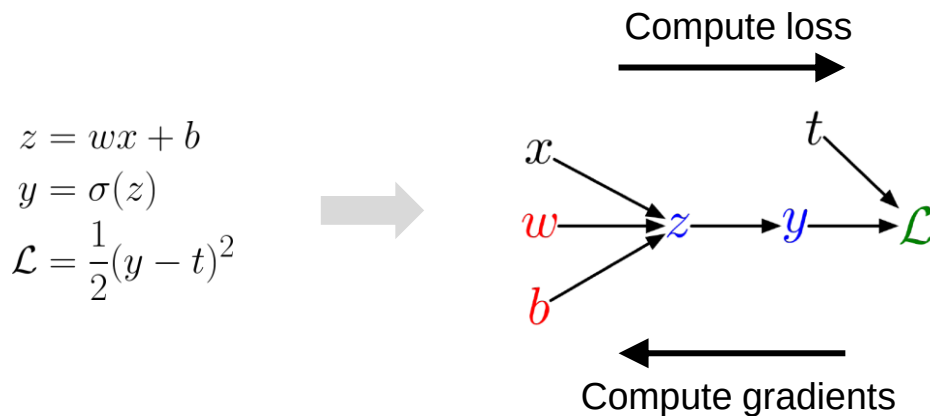
Lecture 4

Convolutional Neural Networks I: Basics

Ying CAO
SIST, ShanghaiTech
Spring, 2026

Backpropagation

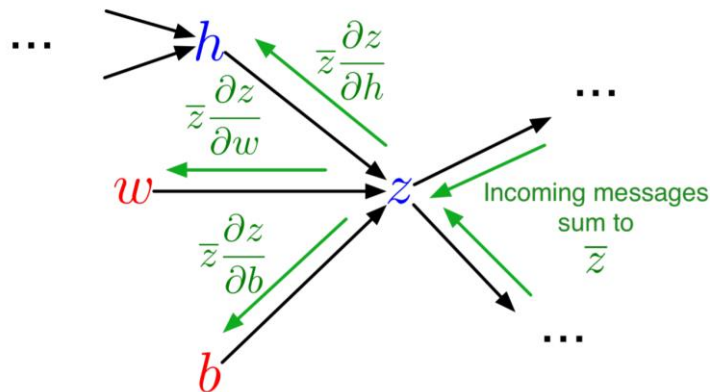
- Represent the computations using a computation graph
 - Nodes: inputs & parameters & computed quantities
 - Edges: a node is computed directly as a function of which other nodes



- Compute loss (forward) and gradients (backward) on the graph

Backpropagation

■ Backprop as message passing:



$$\bar{z} = \frac{\partial L}{\partial z} \quad (L \text{ is the final node})$$

- Each node receives a set of messages from its children, which are aggregated into the gradient of the node; then the node passes messages to its parents
- **Local computation on the graph**: each node only needs to know how to compute gradients w.r.t. its inputs

Vector form of backprop

- Review: Jacobian of vector function $f(x_1, \dots, x_m) = (f_1, \dots, f_n)$

$$\mathbf{J} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_m} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_m} \end{bmatrix}$$

- Vectorized chain rule

$$\mathbf{x} \in \mathbb{R}^m, \mathbf{y} \in \mathbb{R}^n$$

$$g : \mathbb{R}^m \rightarrow \mathbb{R}^n, \mathbf{y} = g(\mathbf{x})$$

$$f : \mathbb{R}^n \rightarrow \mathbb{R}, z = f(\mathbf{y})$$

$$\frac{\partial z}{\partial \mathbf{x}_i} = \sum_{j=1}^n \frac{\partial z}{\partial \mathbf{y}_j} \frac{\partial \mathbf{y}_j}{\partial \mathbf{x}_i}$$

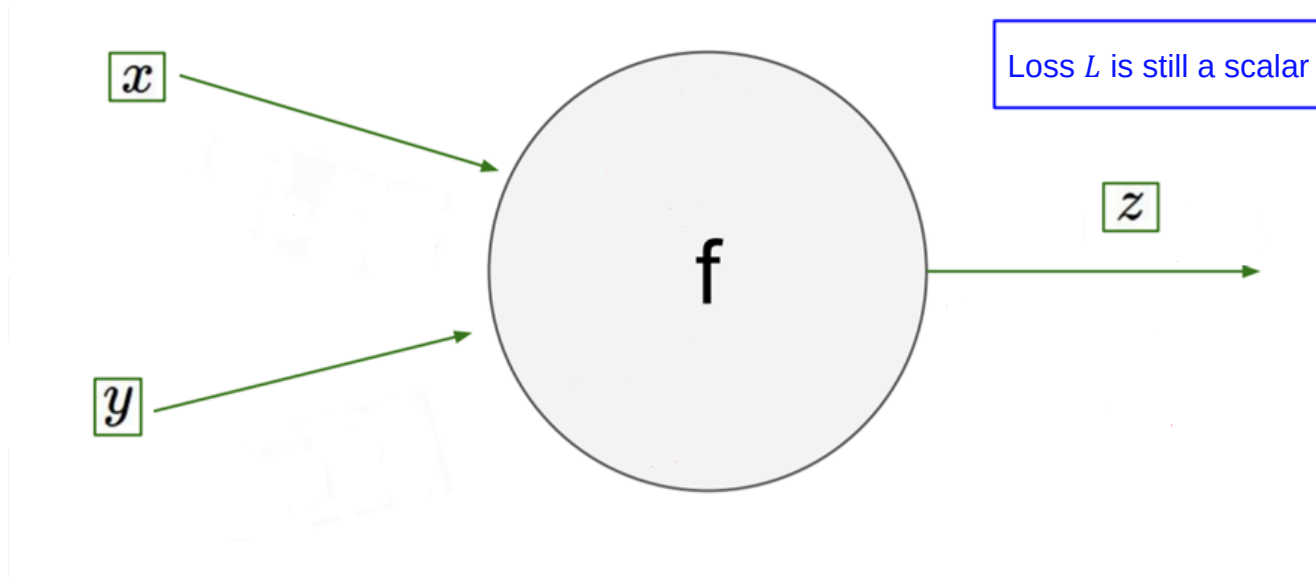
$$\frac{\partial z}{\partial \mathbf{x}} = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}} \right)^T \frac{\partial z}{\partial \mathbf{y}}$$



Jacobian matrix of size $n \times m$

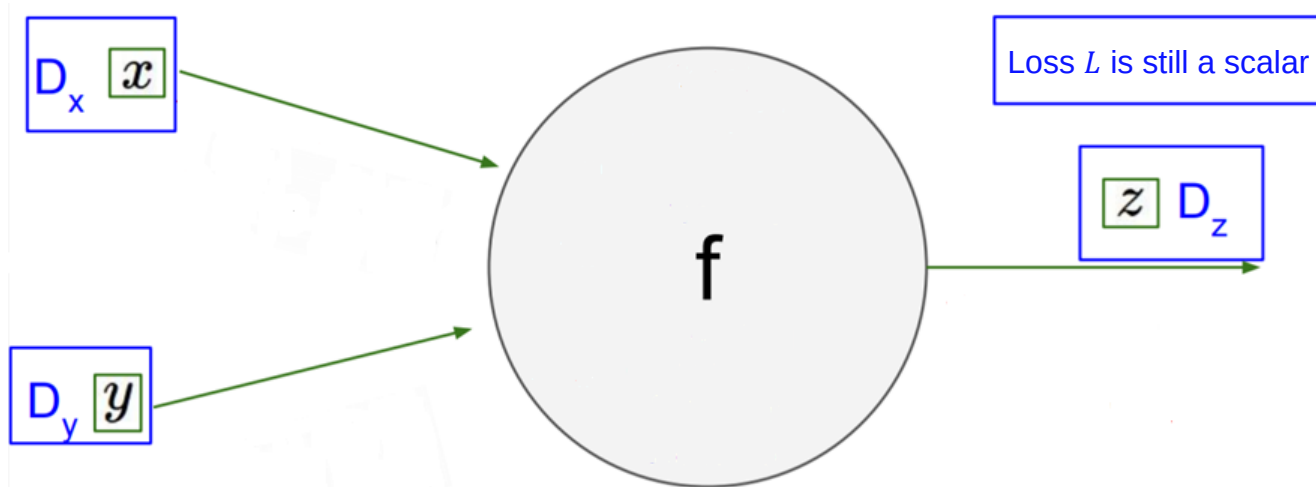
Vector form of backprop

- Forward pass with vectors



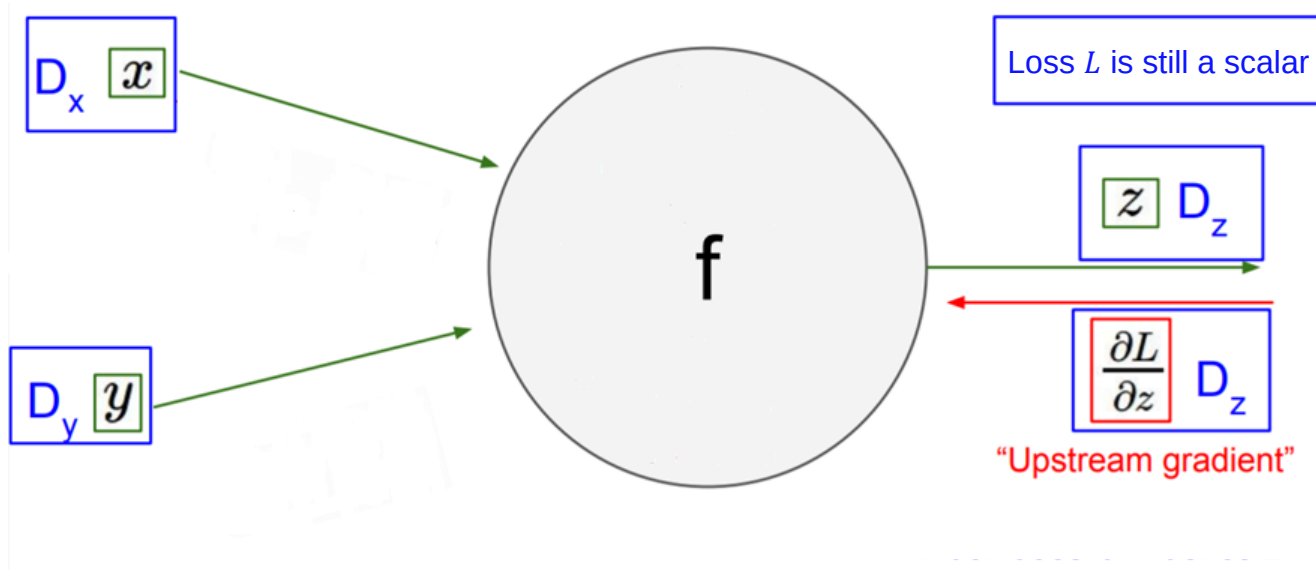
Vector form of backprop

- Backward pass with vectors



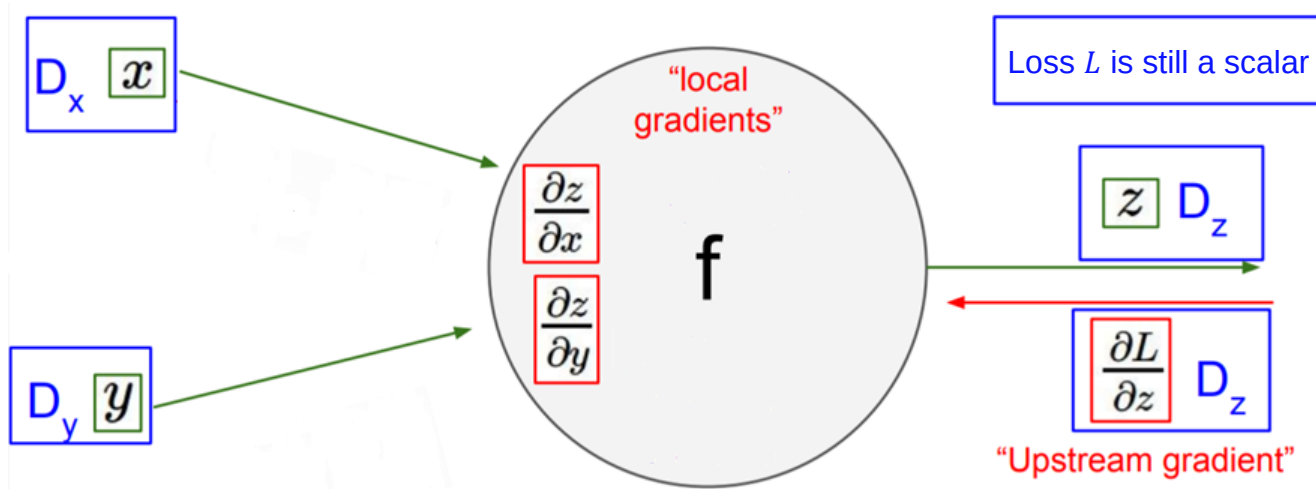
Vector form of backprop

- Backward pass with vectors



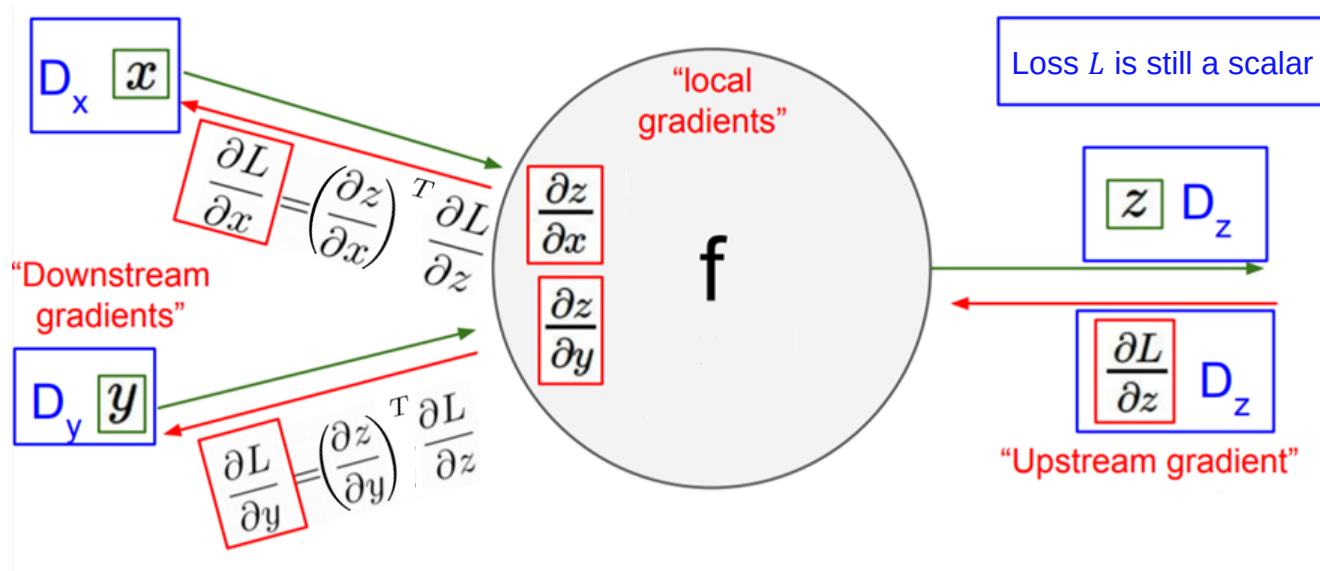
Vector form of backprop

- Backward pass with vectors



Vector form of backprop

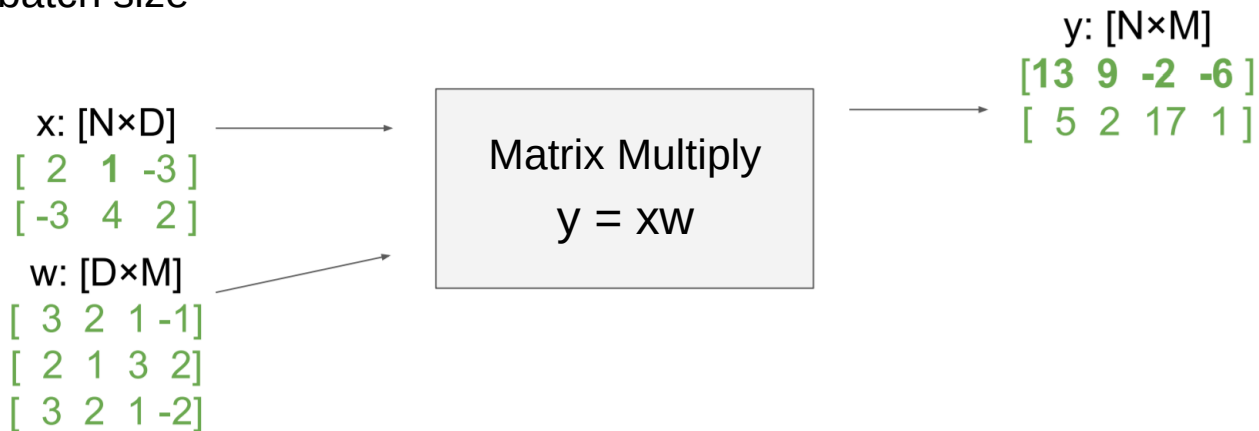
- Backward pass with vectors



Matrix form of backprop

- Often used for mini-batches of examples

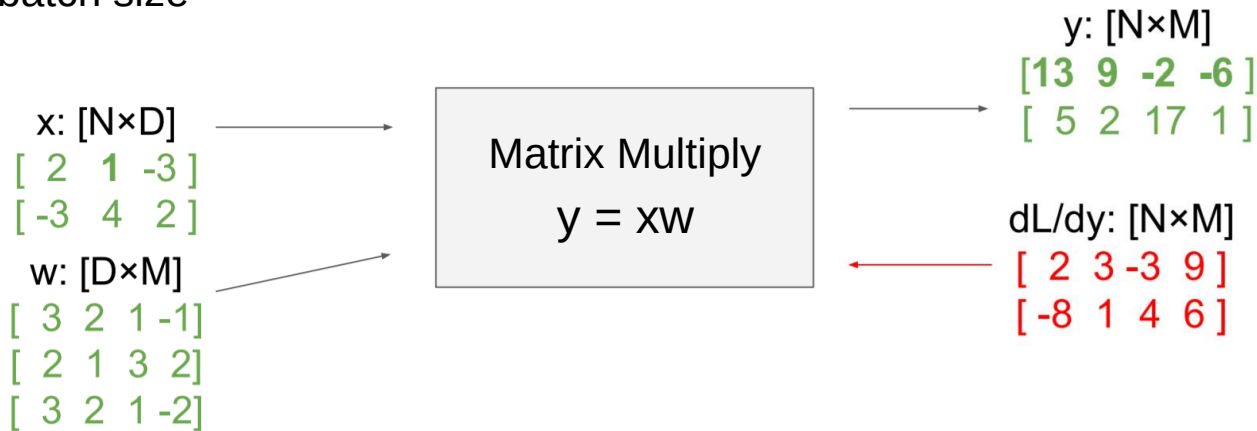
- N is batch size



Matrix form of backprop

- Often used for mini-batches of examples

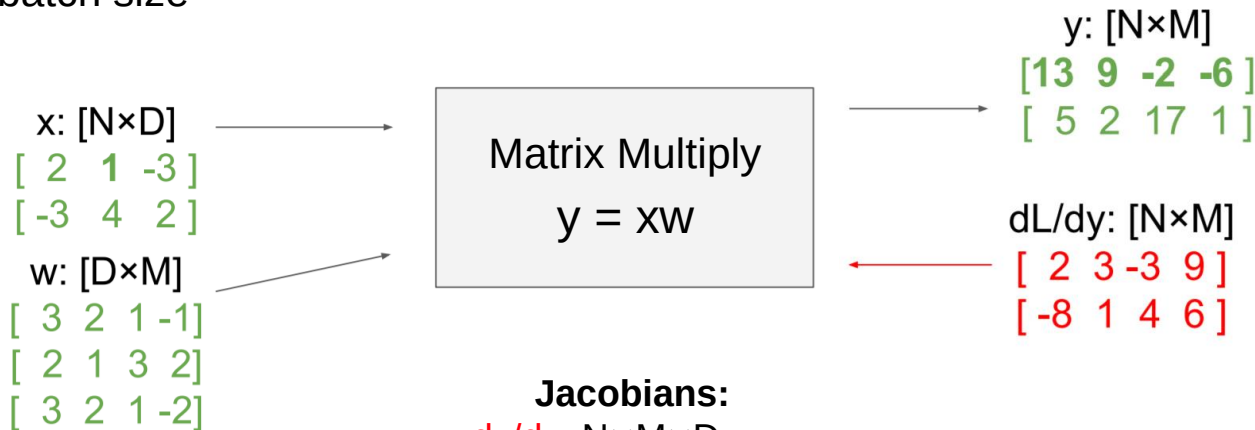
- N is batch size



Matrix form of backprop

- Often used for mini-batches of examples

- N is batch size



Jacobians:

$dy/dx: N \times M \times D$

$dy/dw: N \times M \times (D \times M)$

For a neural net, we may have

$N=64, D=M=4096$

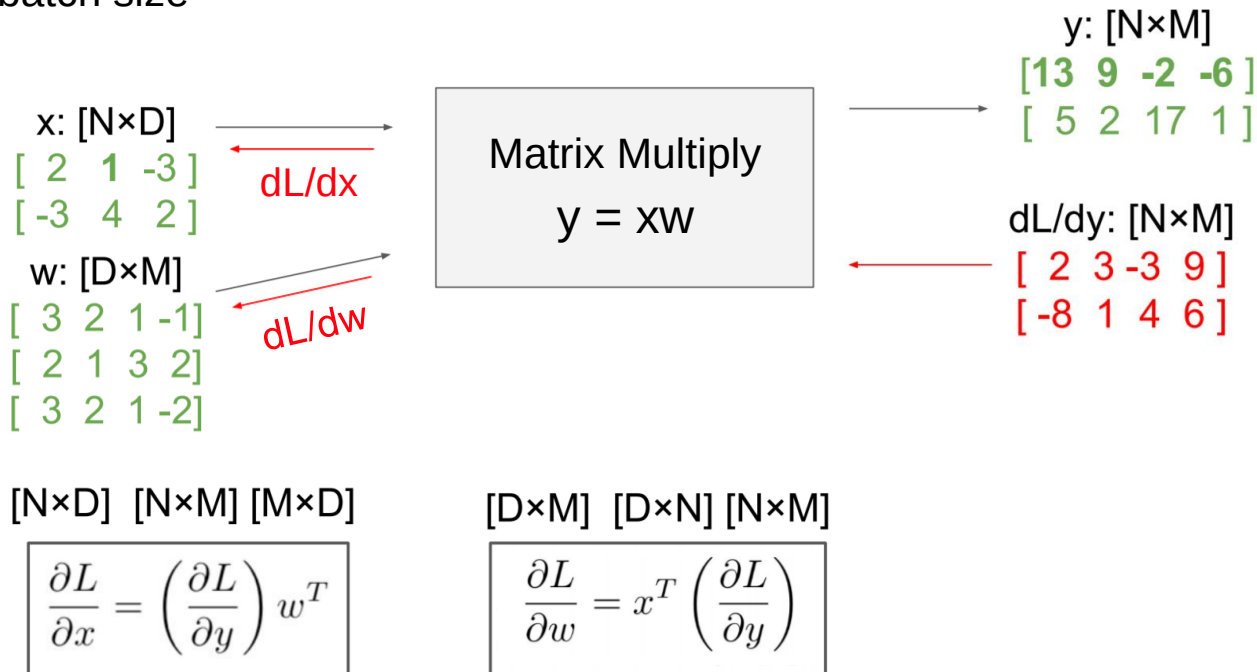
Jacobians take up a lot of memory!

Must work with them implicitly!

Matrix form of backprop

- Often used for mini-batches of examples

- N is batch size



Outline

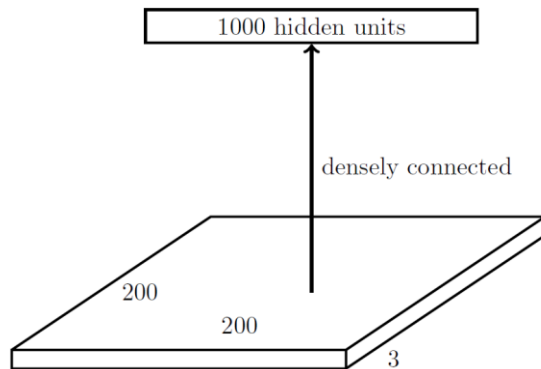
- Why convolutional neural networks (CNNs)?
 - Motivation and overview
- What is CNN?
 - Convolution layers
 - Pooling layers
 - Closer look at activation functions
 - Math properties

Acknowledgement: Roger Grosse's CSC321 and Feifei Li et al.'s CS231n notes

Motivation

■ Visual recognition

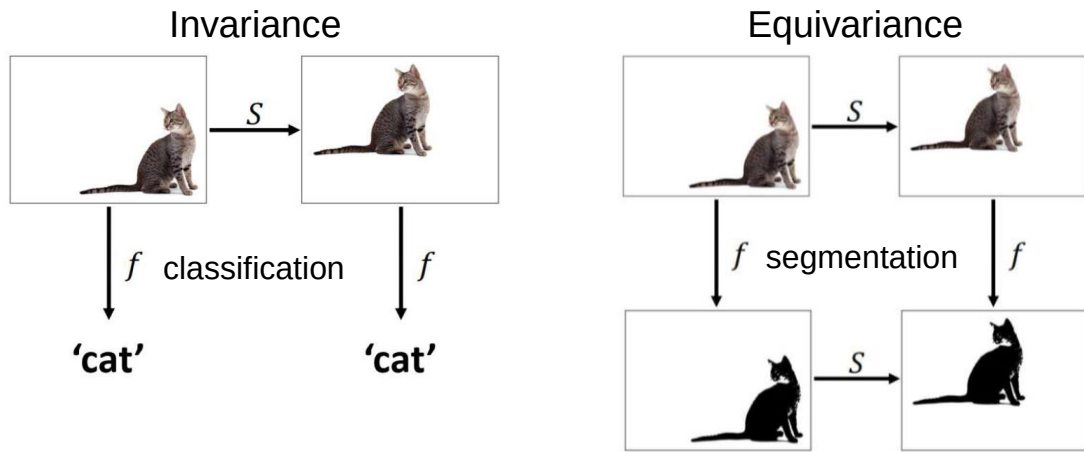
- Suppose we aim to train a network that takes a 200x200 RGB image as input



- What are problems with full connections in the first layer?
 - Too many parameters! $200 \times 200 \times 3 \times 1000 = 120$ million
 - What happens if the object in the image shifts a little?

Our goal

- Visual understanding: design a neural network that
 - Can deal with very **high-dimensional inputs**
 - Can exploit **local structures** in images (2D grids of pixels)
 - Can achieve **invariance/equivariance to certain variations**, e.g., translation, deformation



From Bernhard Kainz's deep learning notes

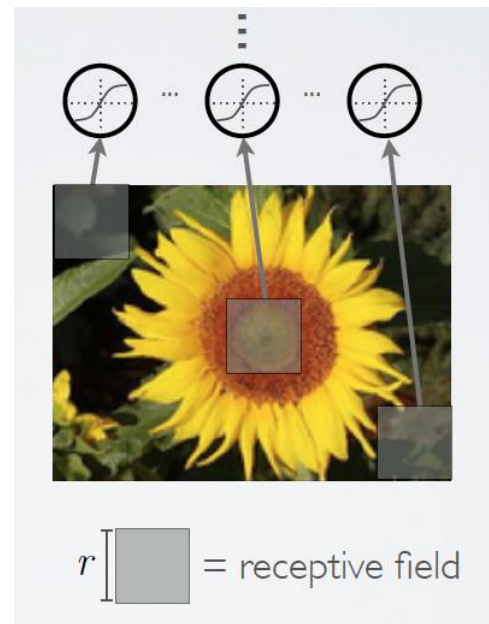
Our goal

- Visual understanding: design a neural network that
 - Can deal with very **high-dimensional inputs**
 - Can exploit **local structures** in images (2D grids of pixels)
 - Can achieve **invariance/equivariance to certain variations**, e.g., translation, deformation
- Convolutional neural networks (CNNs) leverage these ideas
 - Local connectivity
 - Weight sharing
 - Pooling

Overview of CNN

■ First idea: Local connectivity

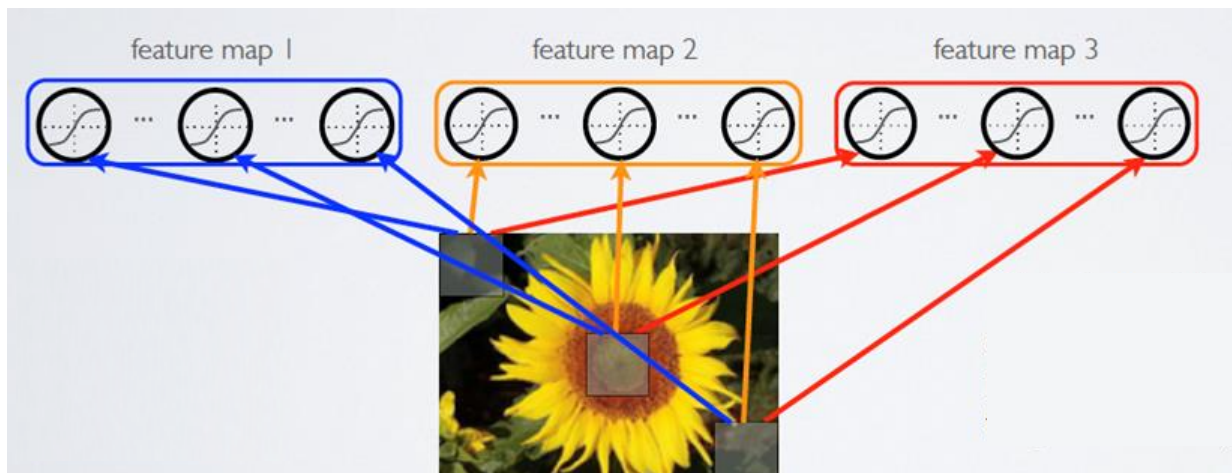
- Each neuron is connected only to a local region of the input
- Each neuron has a local receptive field
- Usually it is connected to all channels



Overview of CNN

■ Second idea: Weight sharing

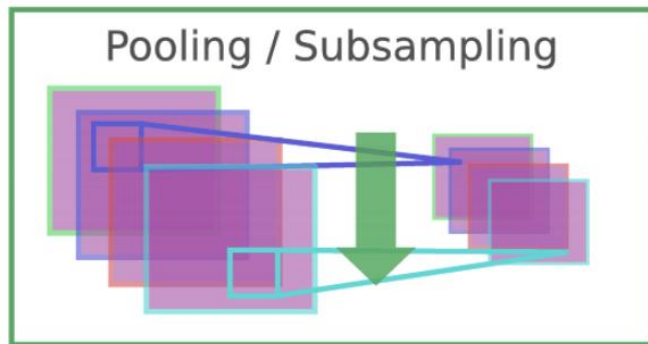
- Neurons in a layer are organized into a stack of “feature maps”
- Neurons in a feature map share the same weights, but connected to different local regions in the input



Overview of CNN

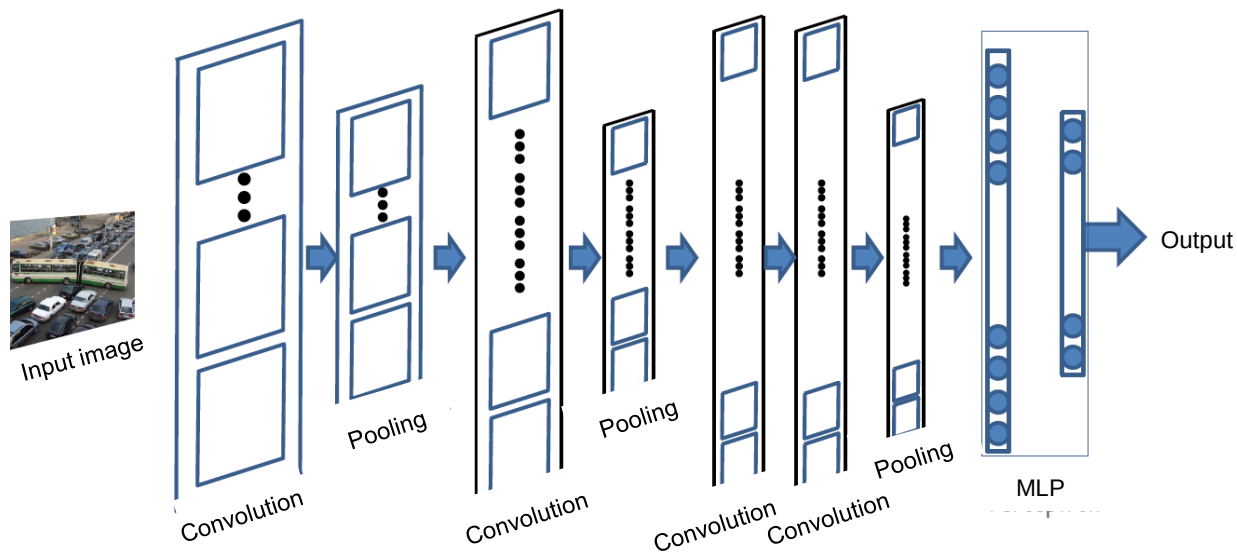
■ Third idea: Pooling

- Pool neighboring neurons; discard location information in a small region
- Less sensitive to small translation/deformation by ignoring spatial details
- Reduce the spatial size of feature maps (subsampling)



Overview of CNN

- Interleave convolution (the 1st & 2nd ideas) and pooling operations (the 3rd idea)

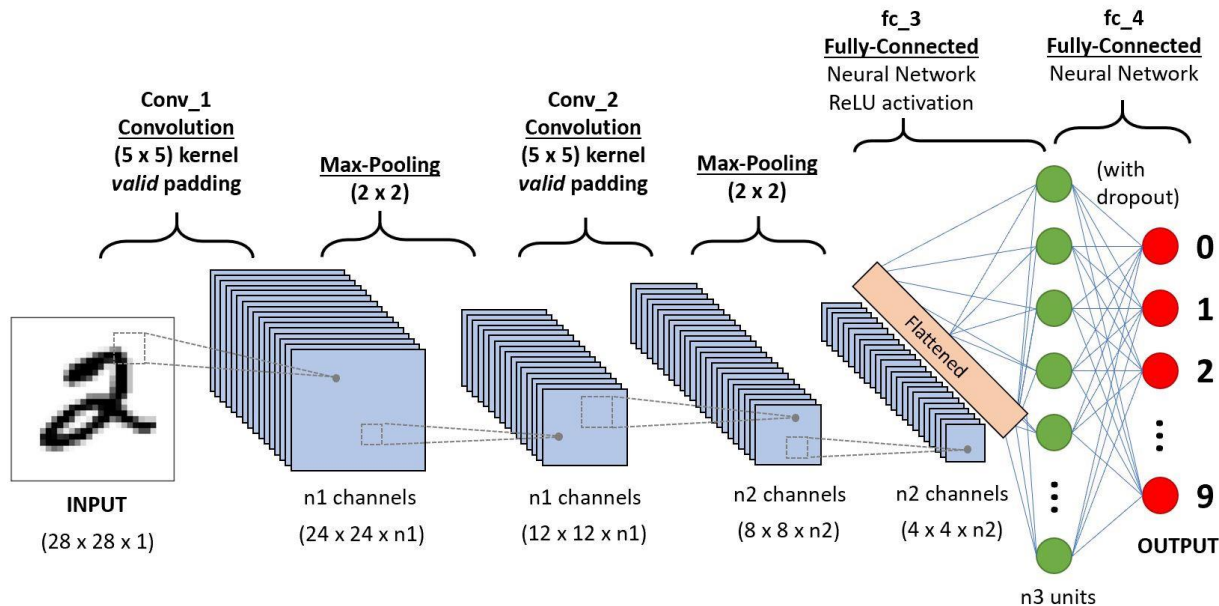


Convolution (linear) is followed by activation function (non-linearity)

Overview of CNN

■ LeNet-5 (1998)

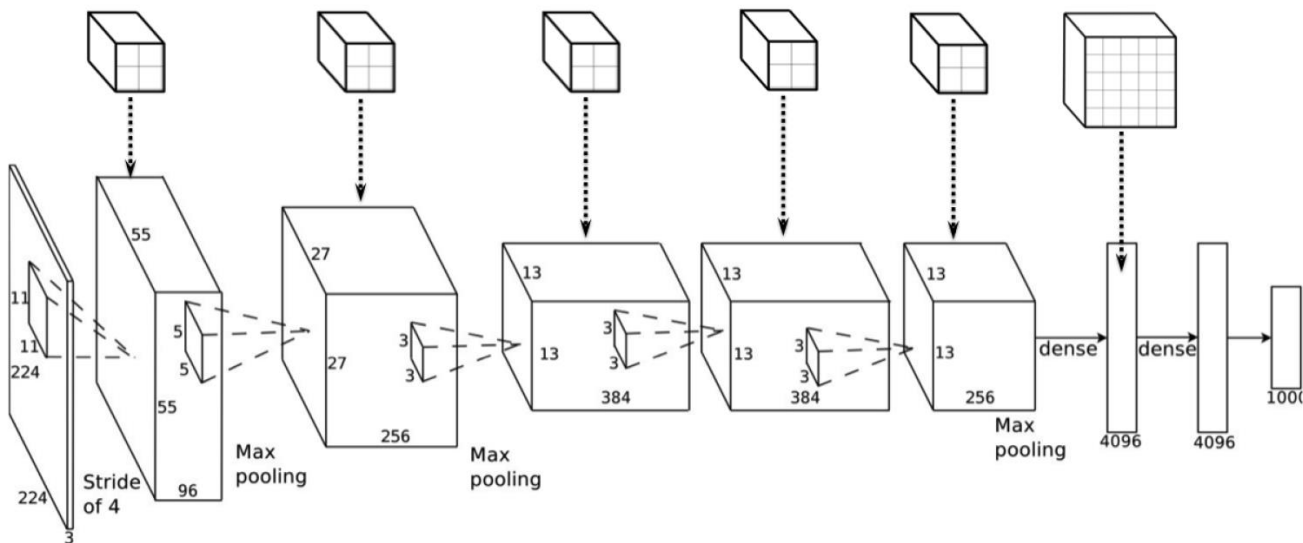
- Handwritten digit recognition



Overview of CNN

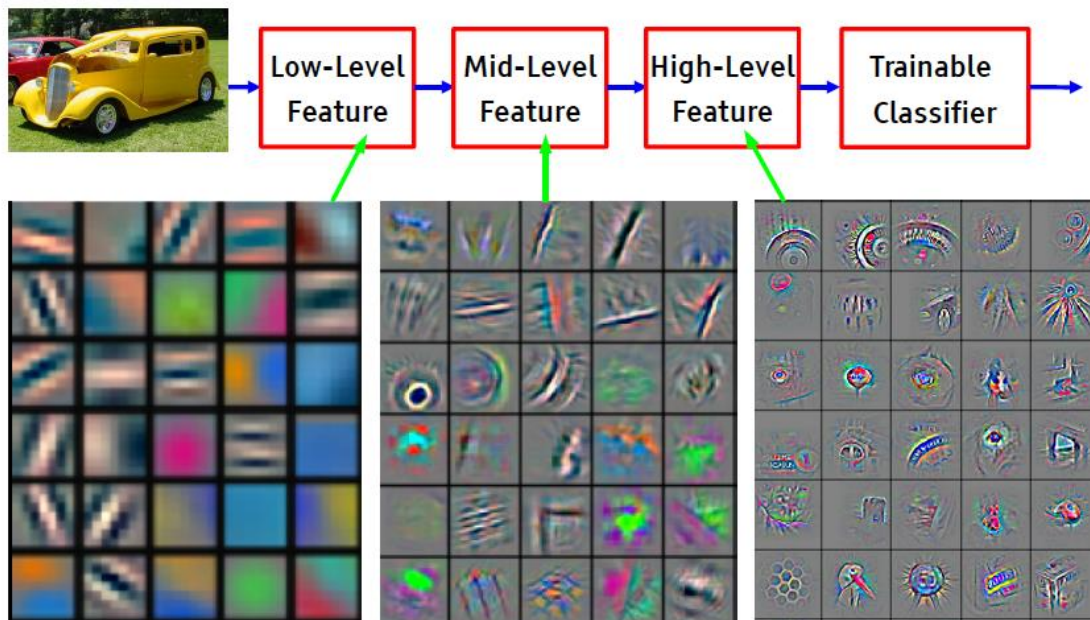
■ AlexNet (2012)

- Deeper network for image recognition



Overview of CNN

- Extract hierarchical features for images



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus, 2013]

Outline

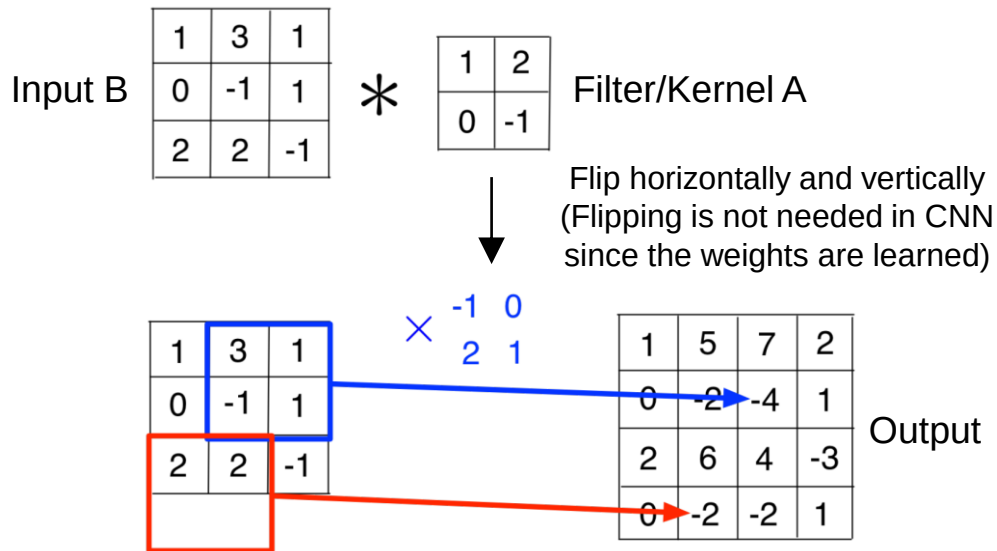
- Why convolutional neural networks (CNNs)?
 - Motivation and overview
- What is CNN?
 - Convolution layers
 - Pooling layers
 - Closer look at activation functions
 - Math properties

Acknowledgement: Roger Grosse's CSC321 and Feifei Li et al.'s CS231n notes

2D Convolution

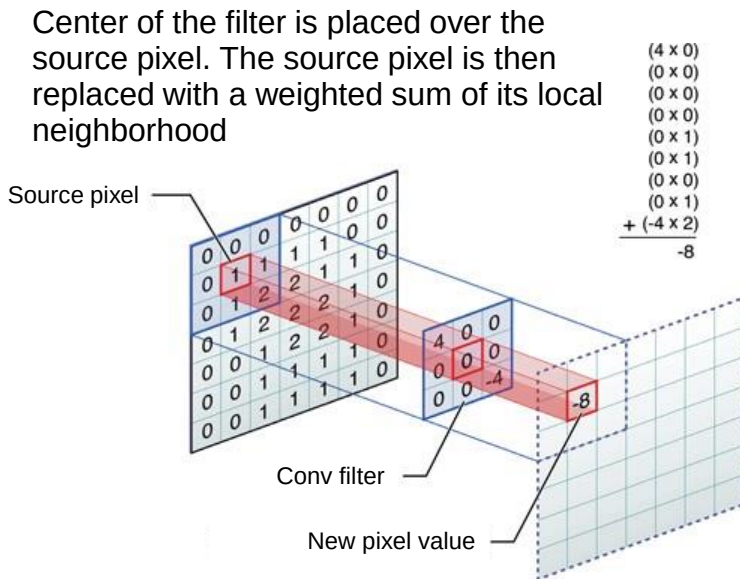
- If A and B are two 2D arrays, then:

$$(A * B)_{ij} = \sum_s \sum_t A_{st} B_{i-s, j-t}$$



2D Convolution

- Slide the filter across the input spatially and compute weighted sums (dot products)



1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Input

4		

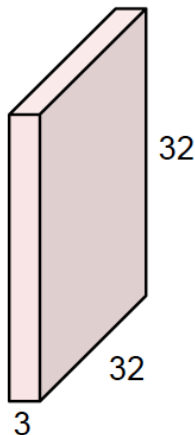
Output

Convolution layer

- What does a convolution layer do?

- Convolve filters with the input

32x32x3 image



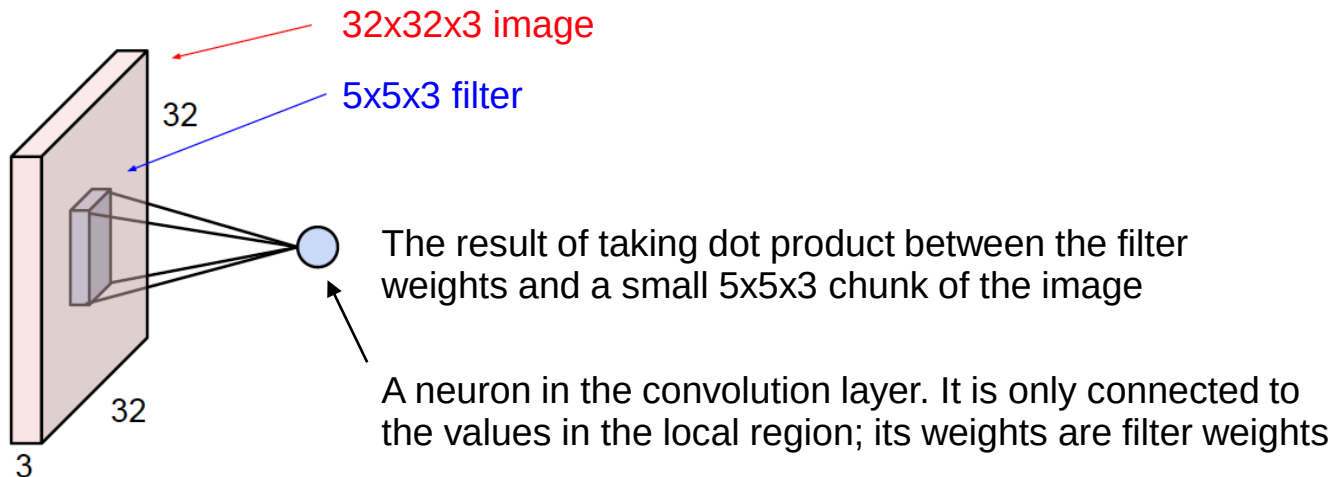
5x5x3 filter



Consider a filter of size 5x5
(5x5x3 weights since the filter is
applied to all input channels)

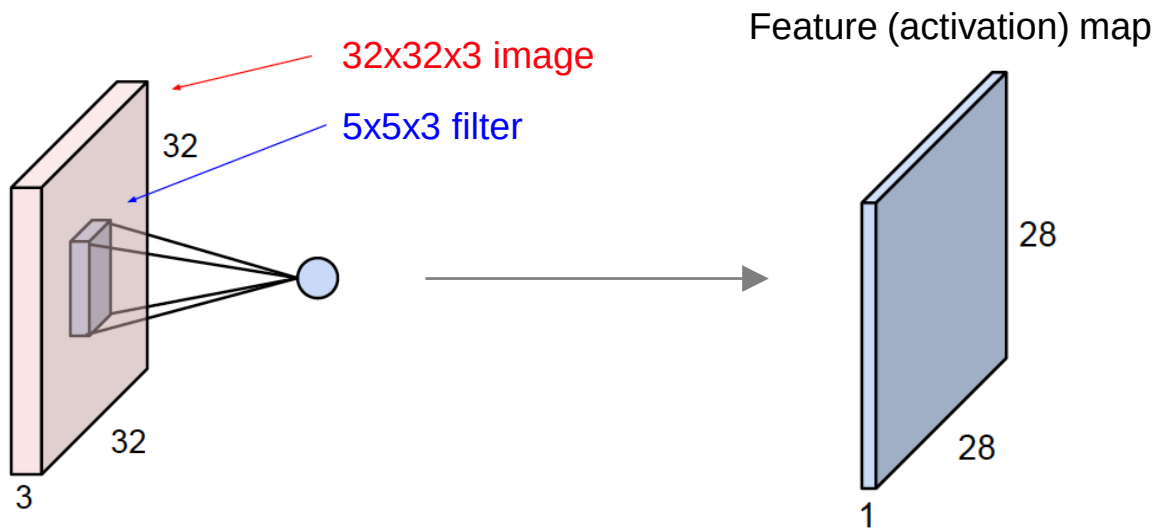
Convolution layer

- Apply the filter to a local input region
 - Local connectivity: a neuron only observes a part of the input



Convolution layer

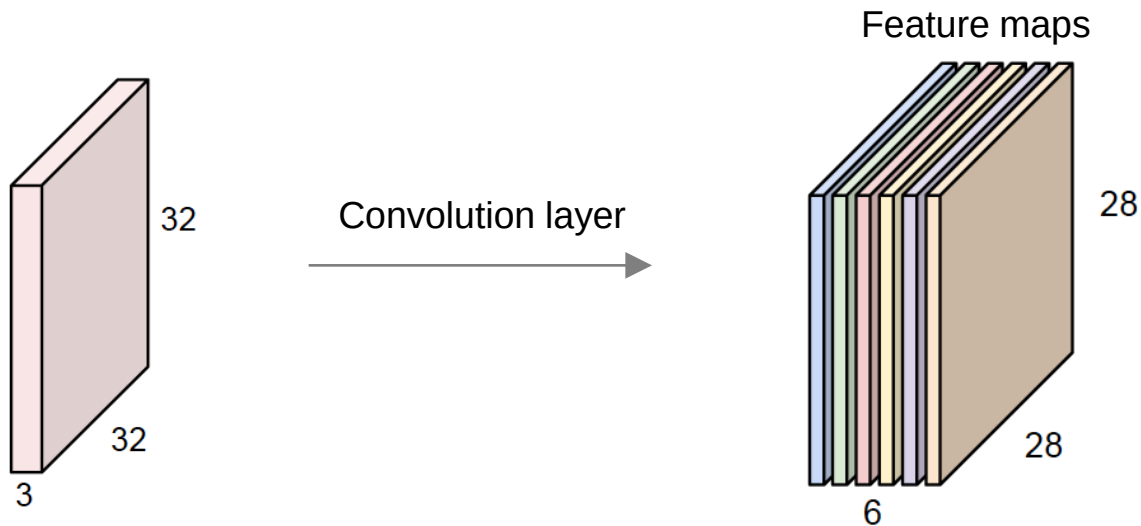
- Slide the filter over spatial locations
 - Weight sharing: all neurons in a feature map share the same filter weights



Convolution layer

- Use multiple filters

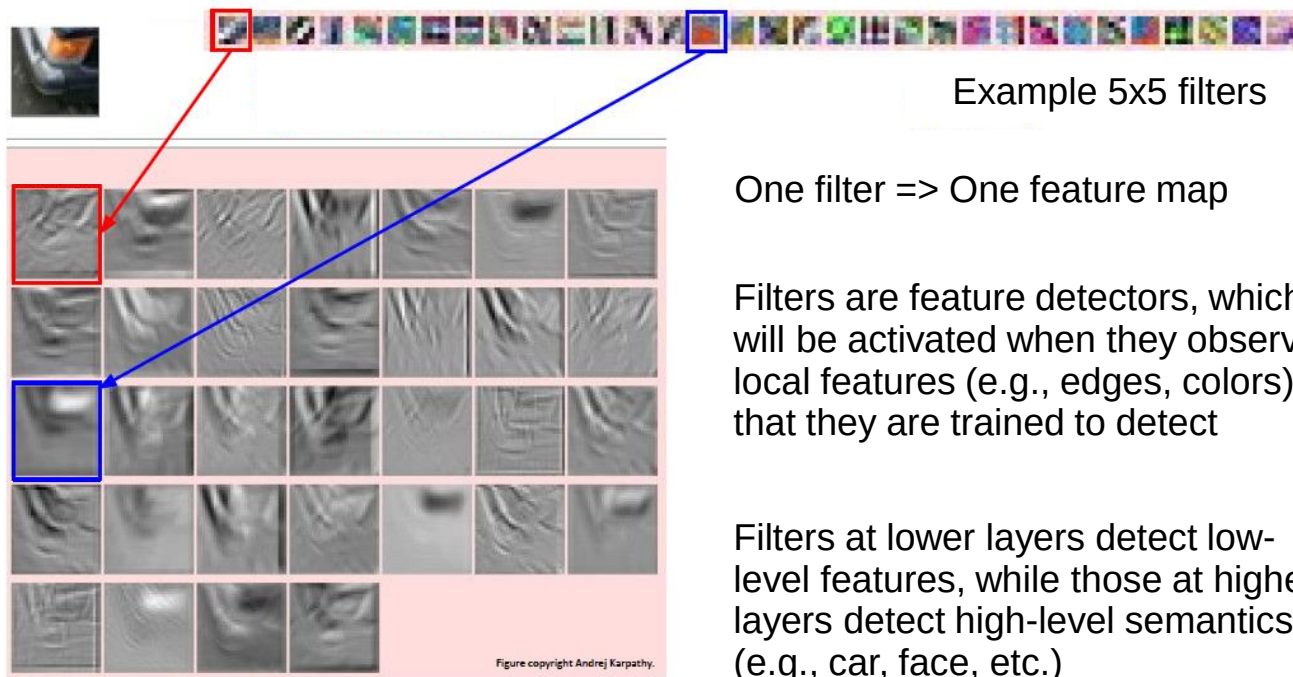
If there are 6 5x5 filters, 6 separate feature maps will be obtained



Stack them up to get a “new image” of size 28x28x6

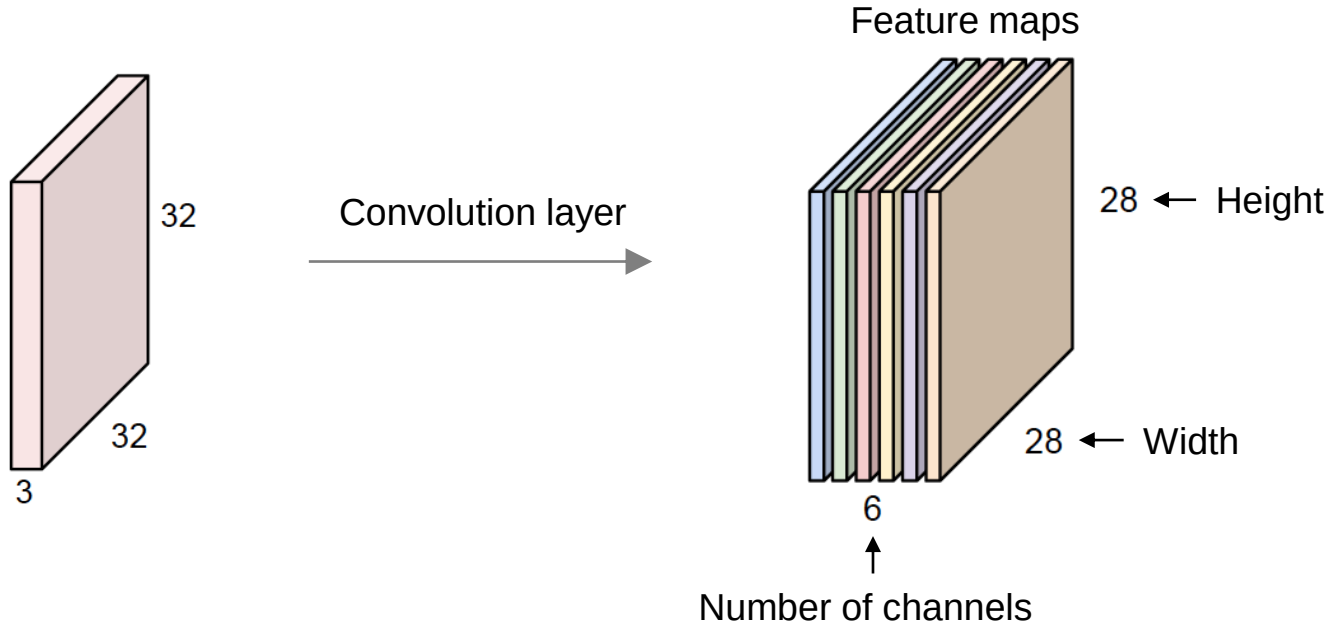
Convolution layer

■ Visualizing filters and their feature maps



Convolution layer

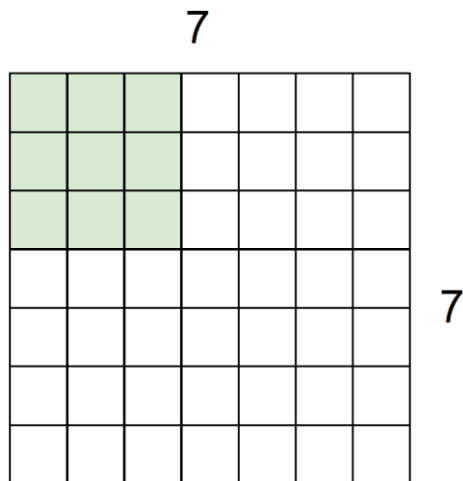
- What is output size?
- How many learnable parameters?



Convolution layer

■ Output size

- Stride = Number of units by which a filter moves across the input



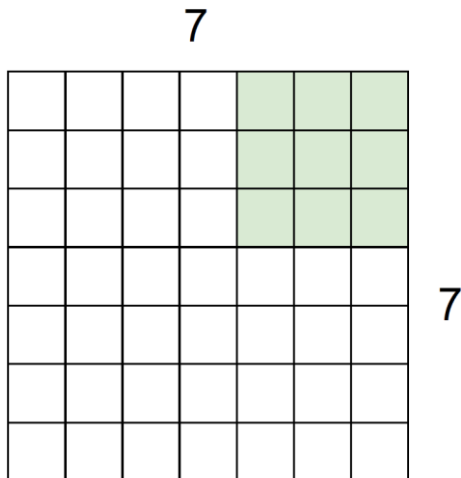
7x7 input
3x3 filter

Stride = 1

Convolution layer

■ Output size

- Stride = Number of units by which a filter moves across the input



7x7 input
3x3 filter

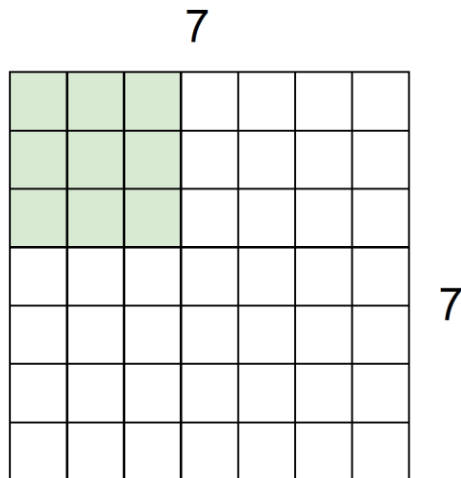
Stride = 1

=> **5x5 output**

Convolution layer

■ Output size

- Stride = Number of units by which a filter moves across the input



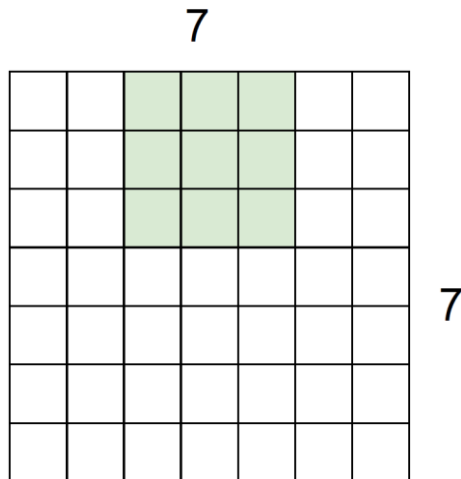
7x7 input
3x3 filter

Stride = 2

Convolution layer

■ Output size

- Stride = Number of units by which a filter moves across the input



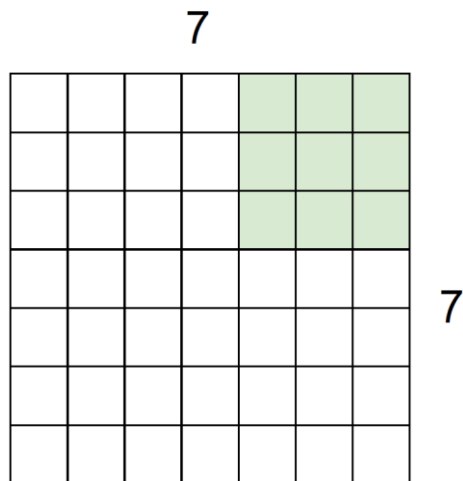
7x7 input
3x3 filter

Stride = 2

Convolution layer

■ Output size

- Stride = Number of units by which a filter moves across the input



7x7 input
3x3 filter

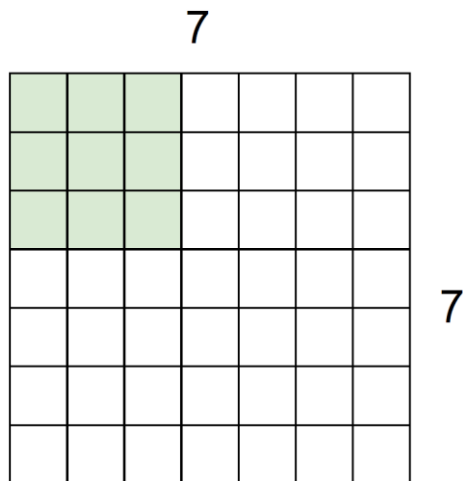
Stride = 2

=> **3x3 output**

Convolution layer

■ Output size

- Stride = Number of units by which a filter moves across the input



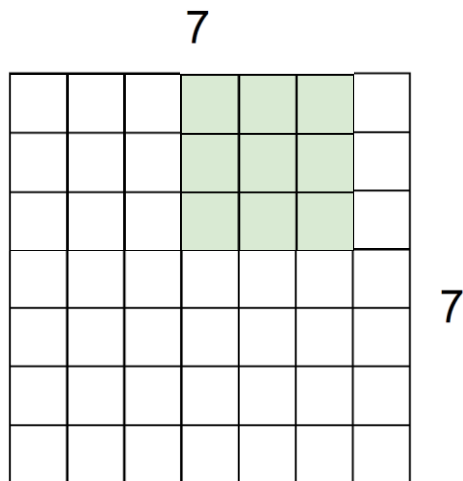
7x7 input
3x3 filter

Stride = 3

Convolution layer

■ Output size

- Stride = Number of units by which a filter moves across the input



7x7 input
3x3 filter

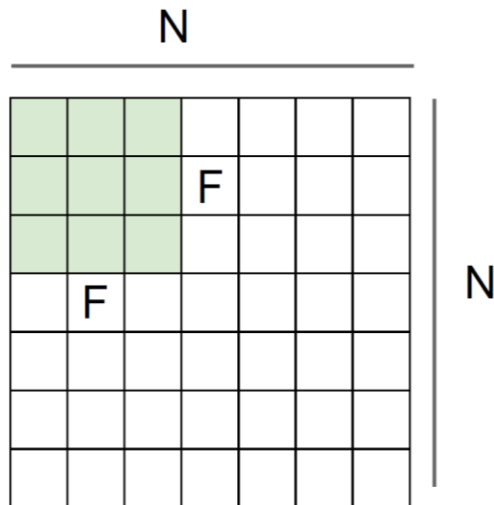
Stride = 3

Doesn't fit

Convolution layer

■ Output size

- Stride = Number of units by which a filter moves across the input



$$\text{Output size} = (N - F) / \text{stride} + 1$$

N=7, F=3:

$$\text{Stride 1} \Rightarrow (7 - 3) / 1 + 1 = 5$$

$$\text{Stride 2} \Rightarrow (7 - 3) / 2 + 1 = 3$$

$$\text{Stride 3} \Rightarrow (7 - 3) / 3 + 1 = 2.33$$

Convolution layer

■ Zero padding

- Add extra zeros around the borders of the input
- Handle non-integer cases or control output size

0	0	0	0	0	0			
0								
0								
0								
0								

7x7 input
3x3 filter

Stride = 1

Pad = 1

=> 7x7 output

Convolution layer

■ Zero padding

- Add extra zeros around the borders of the input
- Handle non-integer cases or control output size

0	0	0	0	0	0			
0								
0								
0								
0								

7x7 input
3x3 filter

Stride = 1

Pad = 1

=> **7x7 output**

For conv layers with stride 1, filters of size $F \times F$, use zero padding $(F-1)/2$ to preserve input spatial size:

$F=5 \Rightarrow \text{Pad} = 2$

$F=7 \Rightarrow \text{Pad} = 3$

Convolution layer

■ Example:

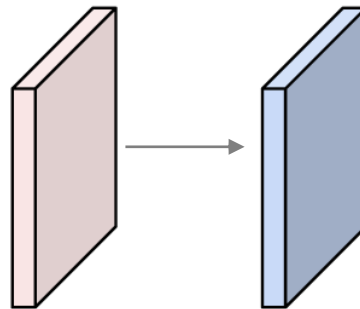
Input size: 32x32x3

10 5x5 filters with stride 1, pad 2

Output size:

$(32 - 5 + 2 \times 2) / 1 + 1 = 32$, so

32x32x10



Convolution layer

■ Example:

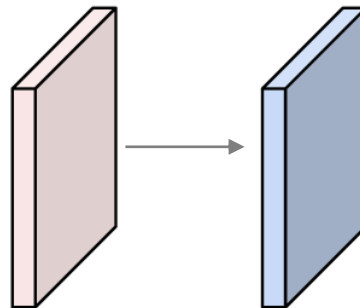
Input size: 32x32x3

10 5x5 filters with stride 1, pad 2

Number of parameters:

Each filter has 5 x 5 x 3 = 75 params

=> 75 x 10 = 750



Convolution layer

■ Summary

- Accept an input tensor of size $W_1 \times H_1 \times D_1$ (D_1 is number of input channels)
- Require four hyperparameters
 - Number of filters C
 - Filter size F
 - Stride S
 - Amount of padding P
- Produce an output tensor of size $W_2 \times H_2 \times D_2$ (D_2 is number of output channels)
 - $W_2 = \frac{W_1 - F + 2P}{S} + 1$
 - $H_2 = \frac{H_1 - F + 2P}{S} + 1$
 - $D_2 = C$
- With parameter sharing, it introduces $F \times F \times D_1$ weights per filter, for a total of $(F \times F \times D_1) \times C$ weights

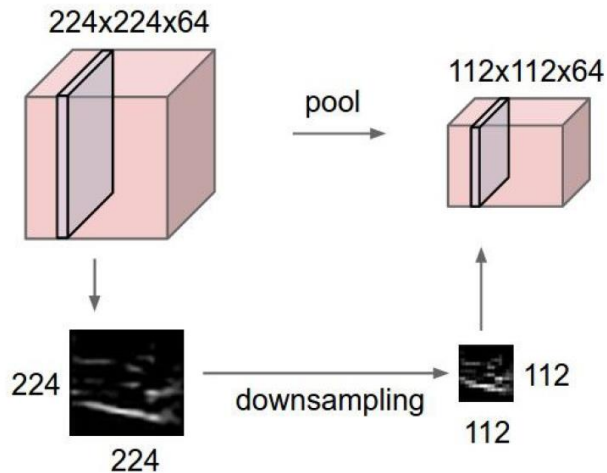
Outline

- Why convolutional neural networks (CNNs)?
 - Motivation and overview
- What is CNN?
 - Convolution layers
 - Pooling layers
 - Closer look at activation functions
 - Math properties

Acknowledgement: Roger Grosse's CSC321 and Feifei Li et al.'s CS231n notes

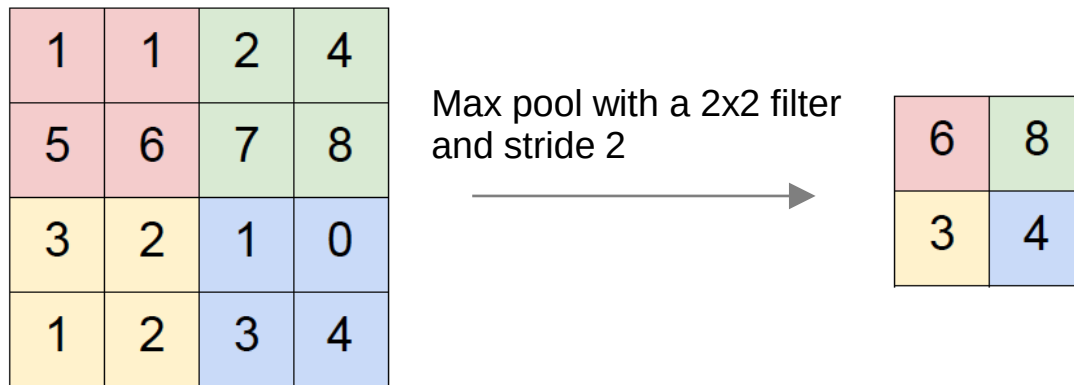
Pooling layer

- Reduce the spatial size of feature maps
 - Perform on each feature map independently
 - No learnable parameters



Pooling layer

- Reduce the spatial size of activation maps
 - Lead to fewer details but lower computational cost
 - Help translation invariance



Pooling layer

■ Summary

- Accept an input tensor of size $W_1 \times H_1 \times D_1$ (D_1 is number of input channels)
- Require two hyperparameters
 - Filter size F
 - Stride S
- Produce an output tensor of size $W_2 \times H_2 \times D_2$ (D_2 is number of input channels)
 - $W_2 = \frac{W_1 - F}{S} + 1$
 - $H_2 = \frac{H_1 - F}{S} + 1$
 - $D_2 = D_1$
- Introduce zero parameters since it computes a fixed function of the input
- Not common to use zero padding for pooling

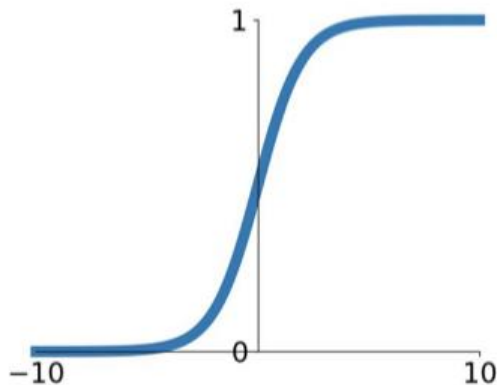
Outline

- Why convolutional neural networks (CNNs)?
 - Motivation and overview
- What is CNN?
 - Convolution layers
 - Pooling layers
 - Closer look at activation functions
 - Math properties

Acknowledgement: Roger Grosse's CSC321 and Feifei Li et al.'s CS231n notes

Activation function

■ Sigmoid



$$\sigma(x) = 1/(1 + e^{-x})$$

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Pro

- Squash numbers to range [0, 1]
- Historically popular since they have nice interpretation as the probability of a neuron firing

Con

- Saturated neurons kill gradients
- $\exp()$ is a bit compute expensive
- Outputs are not zero-centered

Activation function

■ Sigmoid

What happens when the inputs to a neuron are always positive?

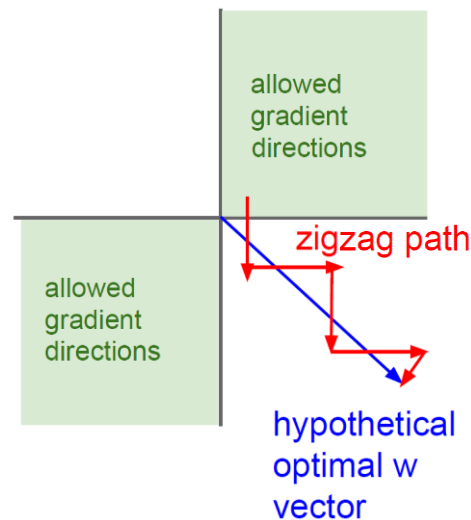
Consider a neuron with input $\{x_i\}$, output y , non-linearity f

$$y = f\left(\sum_i w_i x_i + b\right), \text{ all } x_i > 0$$

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} f'\left(\sum_i w_i x_i + b\right) x_i$$

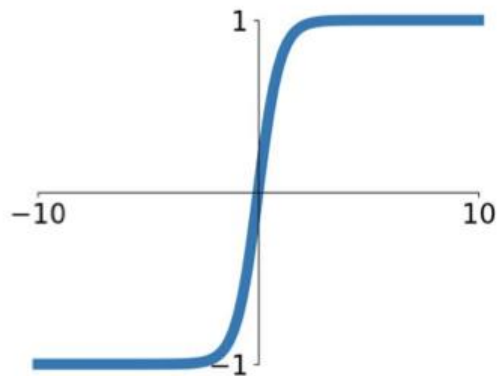
What about gradients w.r.t. $\mathbf{w}$?

Always all positive or all negative $\Rightarrow$ Zigzag updating path



Activation function

■ Tanh



$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\tanh'(x) = 1 - \tanh(x)^2$$

Pro

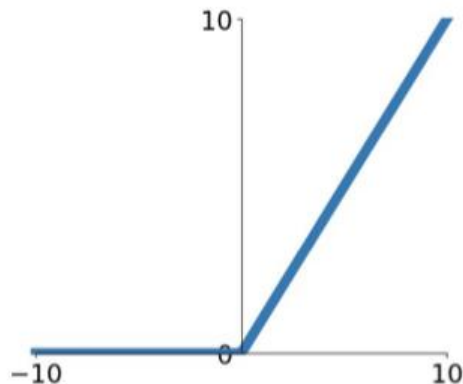
- Squash numbers to range $[-1, 1]$
- Output are zero-centered

Con

- Still kill gradients when saturated

Activation function

■ ReLU (Rectified Linear Unit)



$$f(x) = \max(0, x)$$

Pro

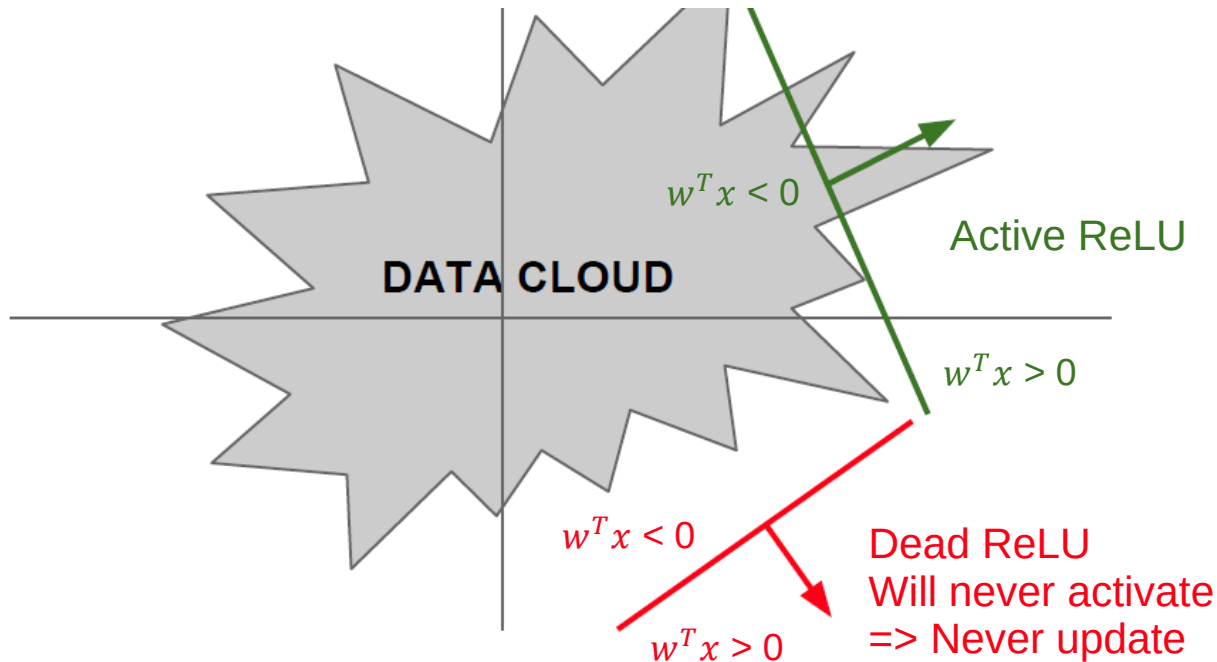
- Do not saturate (in positive input region)
- Very computationally efficient
- Converge much faster than sigmoid/tanh in practice

Con

- Outputs are not zero-centered
- What happens when $x < 0$?

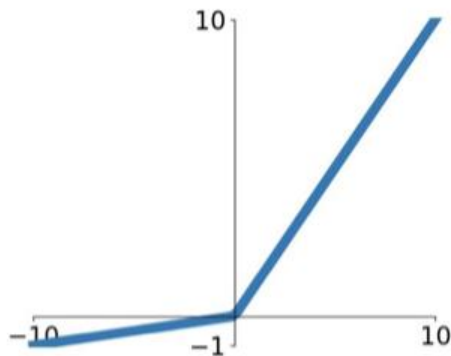
Activation function

- ReLU (Rectified Linear Unit)



Activation function

■ Leaky ReLU



$$f(x) = \max(0.01x, x)$$

Pro

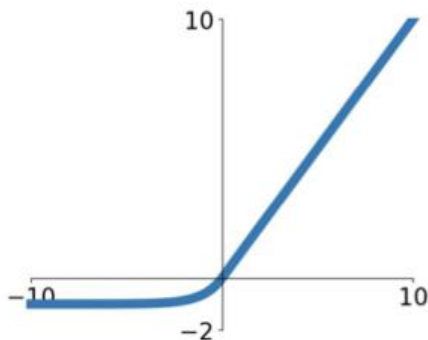
- Do not saturate (in positive input region)
- Very computationally efficient
- Converge much faster than sigmoid/tanh in practice
- **Will not “die”**

Con

- **Outputs are not zero-centered**

Activation function

■ ELU (Exponential Linear Units)



$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$

Pro

- All benefits of ReLU
- Closer to zero-centered outputs
- Negative saturation regime adds some robustness to noise compared with Leaky ReLU

Con

- Computation requires $\exp()$

Summary: Activation function

■ For internal layers in CNNs

- Use **ReLU**
- Try out **Leaky ReLU** / **ELU**
- Don't use **Sigmoid**

■ For output layers

- Task-dependent (e.g., map outputs to $[-1, 1]$)
- Related to loss function (e.g., binary cross entropy expects predictions to be $[0, 1]$)

Outline

- Why convolutional neural networks (CNNs)?
 - Motivation and overview
- What is CNN?
 - Convolution layers
 - Pooling layers
 - Closer look at activation functions
 - Math properties

Acknowledgement: Roger Grosse's CSC321 and Feifei Li et al.'s CS231n notes

Math properties of CNN

- What representations a CNN can capture in general?
- Consider a representation $\phi(\mathbf{x})$ extracted by a CNN ϕ

$$\phi: \mathbf{x} \rightarrow \phi(\mathbf{x}) \in \mathbb{R}^d$$

- Look at how representation $\phi(\mathbf{x})$ changes upon transformations of input $\mathbf{x}$
 - Transformations represent potential variations in images such as translation, scaling, rotation, local deformation, etc.

Math properties of CNN

■ Equivariance

- A representation is equivariant to a transformation g if the transformation of input $\mathbf{x}$ can be transferred to the representation

$$\phi(g(\mathbf{x})) = g(\phi(\mathbf{x}))$$

- Convolution is equivariant to translation



Math properties of CNN

■ Equivariance

- Convolution is equivariant to translation
- Convolution is not equivariant to other rigid transformations, e.g., rotation

`conv2d(`  `,`  `) =` 

Math properties of CNN

■ Invariance

- A representation is invariant to a transformation g if the transformation of input $\mathbf{x}$ has no effect on the representation

$$\phi(g(\mathbf{x})) = \phi(\mathbf{x})$$

- Convolution + **Pooling** is invariant to small translation



Summary of CNN

■ CNN

- ☐ Convolution layers
- ☐ Pooling layers
- ☐ Activation functions detailed
- ☐ CNN properties: equivariance & invariance

■ Next time

- ☐ Training CNNs